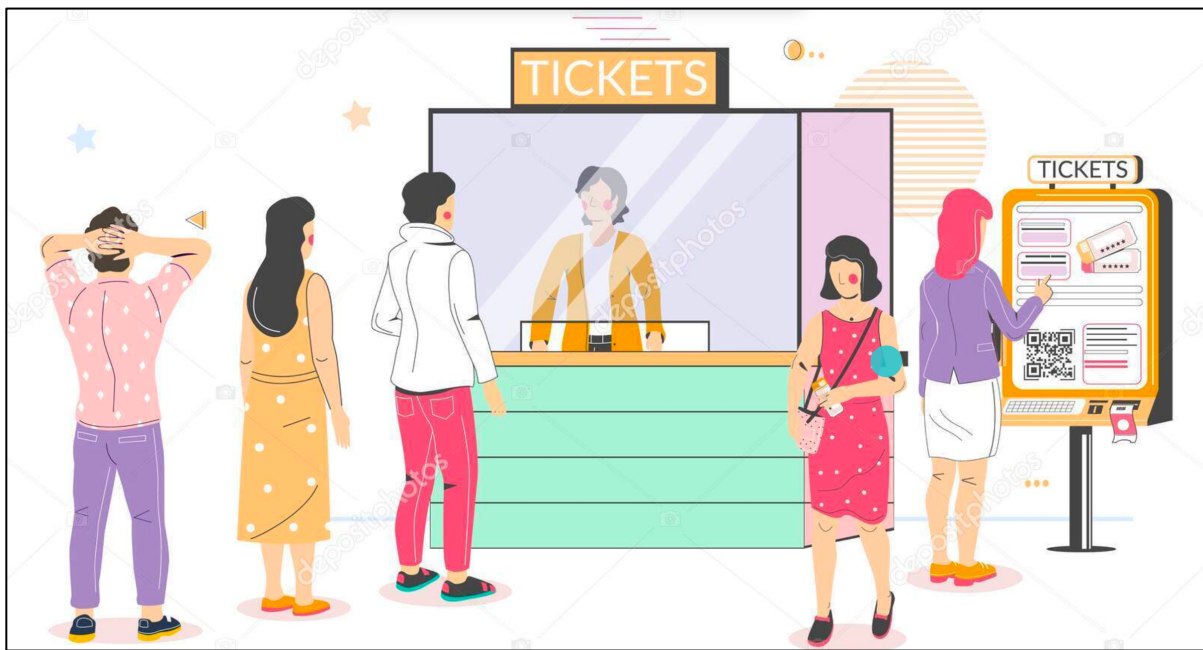


Lab#9 – Test Double

วัตถุประสงค์การเรียนรู้

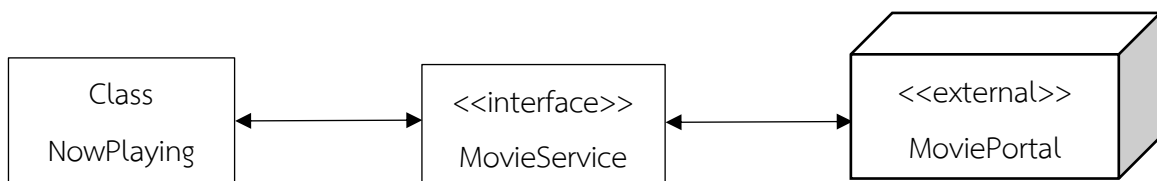
1. ผู้เรียนสามารถวิเคราะห์ Dependencies ใน System under test (SUT) ได้
2. ผู้เรียนสามารถออกแบบการทดสอบระดับ Integration Testing ได้
3. ผู้เรียนสามารถสร้าง Test double ที่เหมาะสมกับลักษณะงานได้
4. ผู้เรียนสามารถเขียน Mock/Stub แทนการทำงานของ Dependencies เพื่อสนับสนุนการทดสอบซอฟต์แวร์ได้



ภาพที่ 1 การซื้อตั๋วชมภาพยนตร์

โรงภาพยนตร์แห่งหนึ่งให้บริการซื้อตั๋วเพื่อเข้าชมภาพยนตร์ 2 รูปแบบ คือ ซื้อผ่านเคาน์เตอร์ให้บริการ (มีพนักงานให้บริการ) และผ่านระบบ Self service (ซื้อด้วยตนเองผ่านเครื่องที่ให้บริการ) ดังแสดงในภาพที่ 1 ทั้งสองรูปแบบทำงานผ่านระบบเดียวกัน ต่างกันตรงที่มีผู้ให้บริการหรือรับบริการด้วยตนเอง ภาพที่ 2 แสดงขั้นตอนการซื้อตั๋วเพื่อเข้าชมภาพยนตร์แบบมีพนักงานให้บริการ ลูกค้าสามารถสอบถามรอบฉายภาพยนตร์ที่สนใจหรือสอบถามเกี่ยวกับภาพยนตร์ที่กำหนดฉายในวันนั้นจากพนักงานได้ด้วย เมื่อลูกค้าแจ้งข้อมูลแก่พนักงานครบถ้วนว่า ต้องการชมภาพยนตร์เรื่องอะไร รอบไหน และต้องการซื้อตั๋วจำนวนเท่าใด พนักงานจะทำการค้นหาที่นั่งที่ว่างอยู่ ณ

ขณะนั้น และแจ้งผังที่นั่งที่ยังว่าง พร้อมกับราคาของที่นั่งแต่ละแบบให้ลูกค้าทราบ เมื่อลูกค้าเลือกที่นั่งและชำระ
เงินเรียบร้อยแล้ว ระบบจะสร้างเลขการจองให้ พนักงานสามารถพิมพ์ตั๋วที่มีเลขการจองที่ตรงกันให้กับลูกค้า โดย
ข้อมูลบนตั๋วชมภาพยนตร์จะปรากฏทั้งเลขการจอง ชื่อภาพยนตร์ รอบ เลขที่นั่ง ราคา เลขสมาชิก (ถ้ามี) และ
บาร์โค้ดเพื่อใช้แสดงต่อพนักงานตรวจตั๋วเมื่อเข้าสู่โรงภาพยนตร์

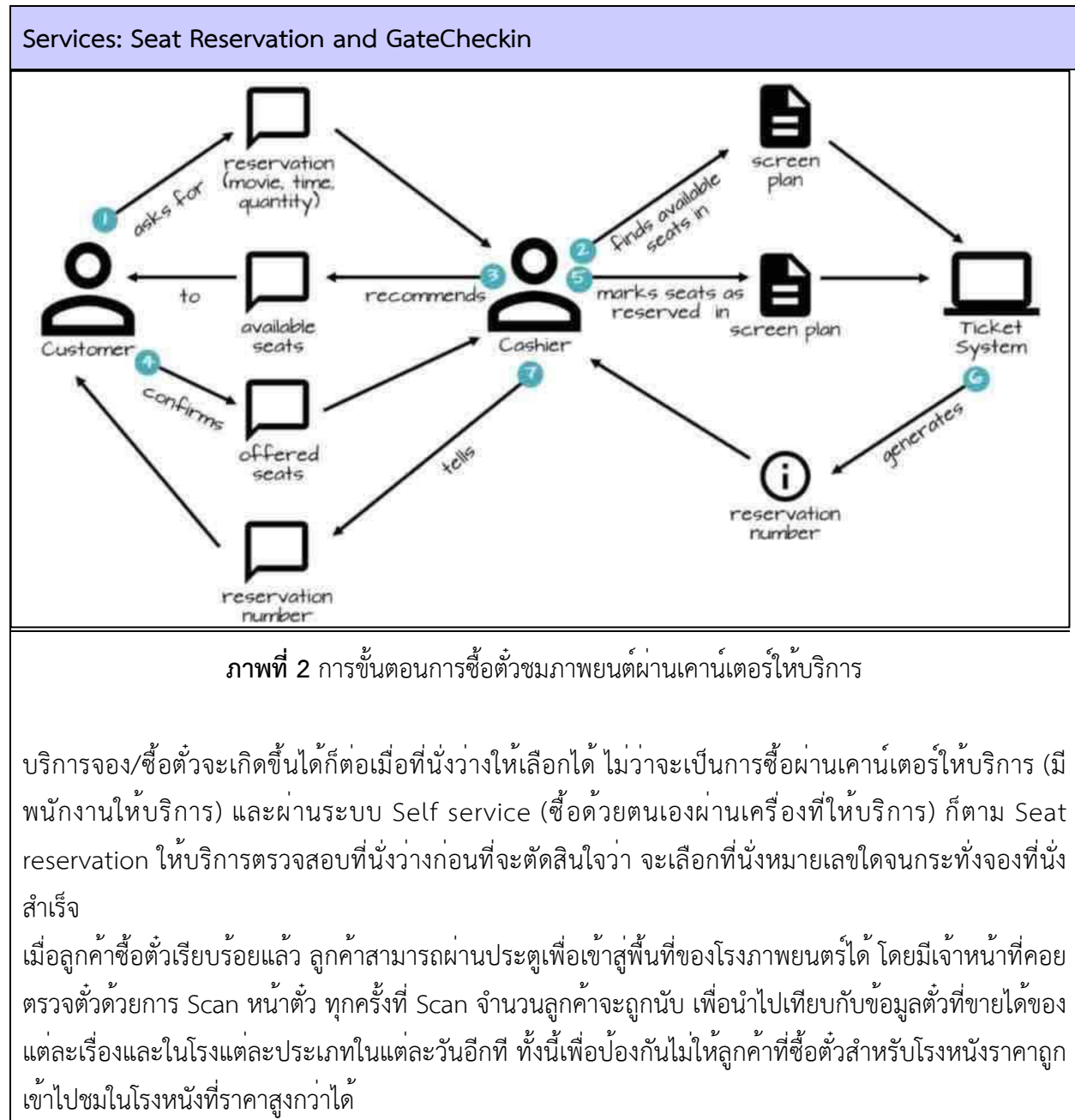
Service: Playlist

MovieService ทำหน้าที่ในการส่ง Request รายการภาพยนตร์เมื่อมีการระบุโรงภาพยนตร์ (location) และ
วันที่ (date) ไปยัง MoviePortal โดยระบบจะส่งรายการภาพยนตร์ทั้งหมดที่มีรอบฉายอยู่ ณ โรงภาพยนตร์นั้น
ตามวันที่ระบุ ส่วน Class NowPlaying ทำหน้าที่คัดกรองการแสดงผลเฉพาะที่ต้องการ เช่น เมื่อต้องการให้
แสดงรายการภาพยนตร์ที่ฉายเฉพาะในโรงแบบ IMAX with Laser ก็จะได้แสดงผลเป็น “The Odyssey,
Spider-Man: Brand New Day, The End of Oak Street” หากมีการเปลี่ยนแปลงข้อมูลการคัดกรอง ก็จะ
แสดงผลเป็นรายการภาพยนตร์ที่ฉายในโรงประเภทนั้น ๆ ออกมาแทน

กิจกรรมที่ 9.1: การสร้าง Stub เพื่อใช้ในการทดสอบระดับ Integration Testing

- (1) ศึกษาข้อมูลความต้องการที่กำหนดให้
- (2) เขียน Class NowPlaying และ <<interface>> MovieService ให้สอดคล้องการความต้องการที่กำหนด
- (3) เขียน Stub เพื่อแทนการทำงานของ Dependencies ในระบบ เมื่อต้องการทดสอบการคัดกรองรายการภาพยนตร์ให้แสดงผลเฉพาะ VIP cinema เท่านั้น
- (4) ทำการทดสอบการทำงานของ Class NowPlaying
- (5) Commit งานให้เรียบร้อย แล้ว Push ไปเก็บไว้ใน remote repository ของตนเอง

- (6) สิ่งที่ต้องส่งใน Google classroom คือ Link ไปยัง GitHub Repository ที่เก็บ Source code ของงาน (Grant สิทธิให้อาจารย์และ TA ด้วย หรือเปิดเป็น public repository)
- Java project พร้อม Source code ที่เกี่ยวข้องกับ Lab#9 ทั้งหมด
 - Optional – readme.md หากต้องการใส่คำอธิบายเพิ่มเติม



กิจกรรมที่ 9.2: การสร้าง Mock ด้วย Mockito Framework เพื่อใช้ในการทดสอบระดับ Integration Testing

- (1) ศึกษากระบวนการซื้อตั๋วชมภาพยนตร์ และการทำงานของ Service: Seat Reservation และ Service: GateCheckin แล้วทำการวิเคราะห์หา Dependencies ขององค์ประกอบที่เกี่ยวข้องกับ SUT
- (2) Clone และศึกษา Source code จาก
https://github.com/ChitsuthaCSKKU/SOA/tree/2026/LabAssignment/Lab9_TestDouble
- (3) เขียน Test double ด้วย Mockito เพื่อทดแทน Response จาก Dependencies ในระบบ เมื่อต้องการทดสอบ
 - a. หมายเลขที่นั่งว่าง
 - b. จำนวนผู้เข้าชมภาพยนตร์ที่ผ่านการตรวจตั๋วแล้ว
- (4) ทำการทดสอบการทำงานของ Service: Seat Reservation และ Service: GateCheckin ด้วย JUnit และ Mockito
- (5) สิ่งที่ต้องส่งใน Google classroom คือ Link ไปยัง GitHub Repository ที่เก็บ Source code ของงาน (Grant สิทธิ์ให้อาจารย์และ TA ด้วย หรือเปิดเป็น public repository)
 - a. Java project พร้อม Source code ที่เกี่ยวข้องกับ Lab#9 ทั้งหมด
 - b. Optional – readme.md หากต้องการใส่คำอธิบายเพิ่มเติม